

S01

Seguridad en Aplicaciones

Semana 1 — Fundamentos del Ciclo de Vida de Desarrollo Seguro (SSDLC) y modelos de madurez

Estructura del curso

TAREAS INDIVIDUALES 8 Semanas 1–8 · 10 pts c/u	PROYECTO FINAL Grupal Único trabajo grupal · 20 pts	EVALUACIÓN 100 pts 80 tareas + 20 proyecto	HORAS DE CLASE 30 h 10 sesiones × 3 horas
--	---	--	---

Ponderación del curso

Componente de Evaluación	Punteo	Descripción
Tareas Individuales (8 × 10 pts)	80	Una tarea individual por semana, semanas 1 a 8
Proyecto Final Grupal	20	Único trabajo grupal del curso · entrega semana 10, hasta 4 integrantes

Tareas individuales

T1 Semana 1 — Diagnóstico de madurez AppSec: evaluación de un caso con dos dominios de OWASP SAMM, ubicación de las actividades de seguridad por fase del SSDLC y evidencia del entorno de laboratorio configurado.

T2 Semana 2 — Modelo de amenazas completo de una aplicación (DFD, análisis STRIDE, priorización y casos de abuso) elaborado con OWASP Threat Dragon.

T3 Semana 3 — Rediseño seguro de un módulo inseguro aplicando los principios de diseño seguro y mapeo de al menos cinco requisitos ASVS a decisiones de diseño.

T4 Semana 4 — Remediación de vulnerabilidades de inyección (SQLi, XSS y command injection) en un repositorio dado, con pruebas unitarias que evidencien la corrección.

T5 Semana 5 — Gestión de criptografía y secretos: sustitución de un almacenamiento de contraseñas inseguro por Argon2, migración de secretos a una bóveda y detección de una fuga en el historial de Git.

T6 Semana 6 — Remediación de vulnerabilidades de API (BOLA/BFLA, validación correcta de JWT y limitación de tasa) sobre una API deliberadamente vulnerable.

T7 Semana 7 — Pipeline de SAST + SCA con Semgrep y OWASP Dependency-Check, generación de SBOM (CycloneDX) y política de fallo del build por severidad.

T8 Semana 8 — DAST automatizado con OWASP ZAP más un conjunto de casos de prueba de seguridad derivados de ASVS nivel 1–2, con reporte de hallazgos.

Entregables del Proyecto Final Grupal

Semana 2 — Publicación del enunciado. Formación de grupos y selección de la aplicación (propia o representativa de una organización guatemalteca).

Semana 4 — Entregable 1: Modelo de amenazas y requisitos de seguridad (ASVS) de la aplicación seleccionada (sin calificación).

Semana 7 — Entregable 2: Resultados de SAST/SCA, SBOM y plan de remediación priorizado (sin calificación).

Semana 8 — Entregable 3: Pipeline DevSecOps funcional con puertas de seguridad e informe de cobertura (sin calificación).

Semana 10 — Entrega final: aplicación con controles integrados, reporte de verificación ASVS y presentación grupal con defensa. [20 pts]

Índice

- 1 Idea central y objetivos
- 2 Por qué la seguridad se construye
- 3 Superficie de ataque
- 4 SSDLC: seguridad en cada fase
- 5 AppSec y madurez
- 6 OWASP SAMM y la lógica de la T1
- 7 Shift-Left y otros marcos
- 8 Caso XYZ y actividad integradora
- 9 Cómo entregar la T1
- 10 Cierre y próxima sesión

Idea central de la semana

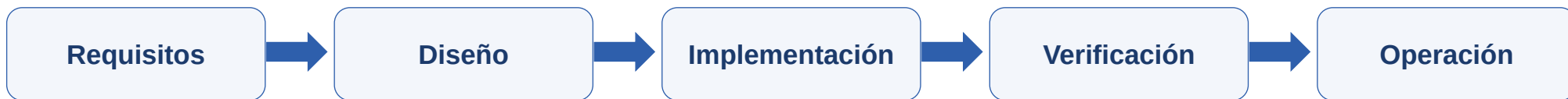
**“La seguridad del software no se agrega al final;
se diseña, se verifica y se mejora
durante todo el ciclo de vida.”**



Idea que acompañará toda la sesión y la T1.

La seguridad se construye, no se agrega

La seguridad del software debe acompañar las decisiones del ciclo de vida, desde los requisitos hasta la operación.



Seguridad = una preocupación transversal, no una etapa aislada.

Competencias de la semana

- 1 Explicar el enfoque shift-left y su impacto en el costo y la calidad de la seguridad del software.
- 2 Situar las actividades de seguridad en cada fase del ciclo de vida de desarrollo.
- 3 Relacionar OWASP SAMM, Microsoft SDL y NIST SSDF como marcos de un programa de AppSec.

¿Por qué la seguridad se construye?

Porque las decisiones de seguridad empiezan antes del código.

- Un requisito ambiguo puede convertirse en un diseño inseguro.
- Un diseño inseguro puede obligar a reescribir componentes.
- Un defecto detectado tarde puede aumentar retrabajo, exposición e impacto.
- La seguridad debe acompañar el ciclo, no esperar a la última prueba.

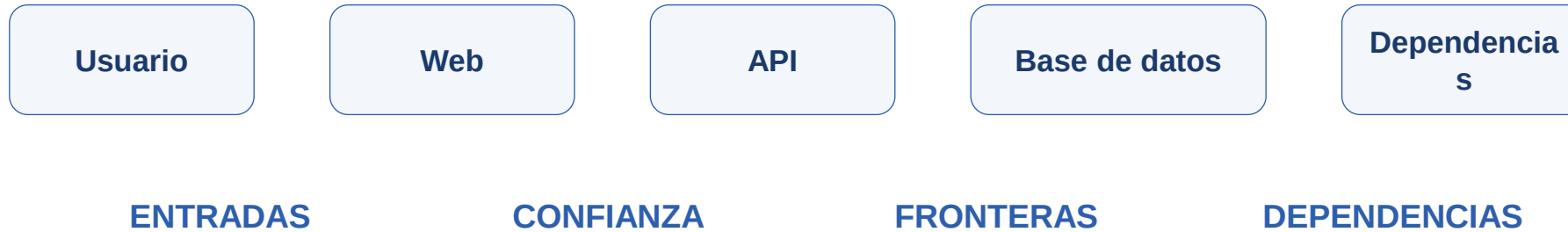
Reto: ¿cuándo preferirías descubrirlo?

Caso: el sistema permite que un usuario consulte reportes de otro usuario.



Pregunta al grupo: ¿qué cambia en cada momento si descubrimos el problema?

¿Dónde puede interactuar o romperse la confianza?



La superficie de ataque no es solo “puertos”: también son datos, APIs, permisos, componentes y supuestos de confianza.

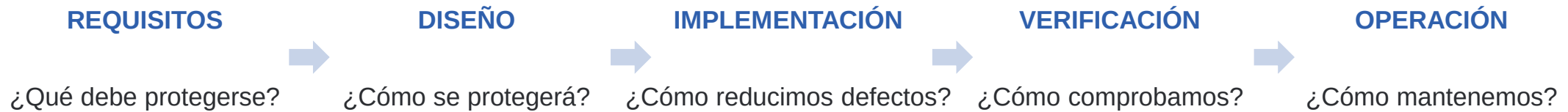
Encuentra 4 puertas

Flujo: Usuario → Web → API → Base de datos

- Señala cuatro lugares donde una entrada podría ser manipulada.
- Identifica un supuesto de confianza que podría romperse.
- Identifica una dependencia externa.
- Comparte una respuesta y explica por qué la consideras parte de la superficie.

Trabajo en parejas · 5 minutos

Seguridad distribuida por el ciclo



La seguridad empieza temprano y continúa durante todo el ciclo.

- Requisitos de seguridad y criterios de aceptación
- Modelado de amenazas y arquitectura segura
- Código seguro, revisión y análisis
- Pruebas y verificación de controles
- Configuración, monitoreo y gestión de vulnerabilidades

Clasifica las actividades

- 1 Definir requisitos de autenticación
- 2 Revisar fronteras de confianza
- 3 Revisar código
- 4 Analizar dependencias
- 5 Ejecutar pruebas de seguridad
- 6 Monitorear eventos

Requisitos

Diseño

Implementación

Implementación / Verificación

Verificación

Operación

No todas las actividades tienen una sola ubicación: lo importante es justificar dónde comienzan y cómo se mantienen.

Actividad, proceso, capacidad y madurez

Actividad

¿Qué hacemos?

Revisar dependencias.

Proceso

¿Cómo lo hacemos?

Existe un procedimiento definido.

Capacidad

¿Podemos hacerlo de forma consistente?
Se aplica de manera repetible.

Madurez

¿Qué tan institucionalizado está?

Se gestiona, mide y mejora.

T1: no basta con detectar “un problema”; debemos interpretar la capacidad que existe.

De la organización a la evidencia

Business Function

Área amplia del modelo

Security Practice

Práctica específica

Actividad / evidencia

Qué se hace o qué demuestra que existe

Diagnóstico

Interpretación del estado observado

Brecha

Qué falta o está débil

Mejora

Qué acción puede fortalecerlo

La regla 2 × 3

Seleccionar 2 Business Functions y analizar sus 3 Security Practices.

Business Function 1

- 1 Security Practice 1
- 2 Security Practice 2
- 3 Security Practice 3

Business Function 2

- 1 Security Practice 1
- 2 Security Practice 2
- 3 Security Practice 3

Resultado: 2 Business Functions × 3 Practices = 6 Practices analizadas

La cadena de razonamiento



Ejemplo: guía de código segura existe, pero no es obligatoria y no hay checklist.

- Evidencia: existe una guía interna, pero su uso no es obligatorio.
- Diagnóstico: la práctica de seguridad está parcialmente formalizada.
- Brecha: no existe un mecanismo uniforme que asegure su aplicación.
- Mejora: incorporar controles verificables al flujo de revisión.

Seguridad más temprano, no seguridad solamente temprano

Mover actividades de seguridad hacia etapas donde las decisiones todavía pueden cambiarse.

Autorización

Descubrir en producción

Definir requisitos + diseño

Arquitectura

Rediseñar al final

Modelar amenazas temprano

Dependencias

Corregir después del despliegue

Revisar durante construcción

Shift-Left NO elimina las pruebas finales: las complementa.

No todos responden la misma pregunta

OWASP SAMM

Madurez de seguridad del software

T1: diagnóstico

NIST SSDF

Prácticas de desarrollo seguro

Proceso de desarrollo

Microsoft SDL

Enfoque de desarrollo seguro

Ciclo con actividades de seguridad

BSIMM

Referencia basada en prácticas observadas

Benchmark de prácticas

En esta semana, SAMM es el marco principal para el diagnóstico de madurez.

NIST SSDF + OWASP Proactive Controls

Dos referencias para reforzar una idea: la seguridad debe integrarse al desarrollo.

NIST SSDF

- Prácticas para integrar seguridad en el ciclo de desarrollo.
- Promueve preparación de la organización, protección del software, producción segura y respuesta.

OWASP Proactive Controls

- Guía práctica de controles que ayudan a prevenir problemas frecuentes.
- Sirve como apoyo para pensar en controles durante el desarrollo.

No son la rúbrica de la T1: son contexto para comprender AppSec.

Security Champions

Una persona dentro del equipo puede ayudar a mantener la seguridad visible durante el desarrollo.

Equipo de desarrollo

Integra seguridad en el trabajo cotidiano.

Security Champion

Conecta al equipo con prácticas y especialistas.

Especialistas AppSec

Aportan experiencia, herramientas y gobierno.

Objetivo cultural: que seguridad no sea responsabilidad exclusiva de “alguien de seguridad”.

Ahora vamos a mirar la aplicación

Sistema de Gestión Académica: Web → API → Base de datos

Equipo	10 desarrolladores · 2 QA · 1 operaciones · 1 PM	CI / pruebas	Pipeline básico · sin SAST/SCA obligatorios · sin DAST obligatorio
Seguridad	Requisitos incompletos · autorización inconsistente · APIs confían en datos del cliente		
		Incidente	Falla de control de acceso en reportes · sin postmortem formal
Dependencias	Actualización manual · sin inventario consolidado · sin SBOM	Operación	Cloud · backups · logs · sin monitoreo centralizado de seguridad

Evidencia → diagnóstico → brecha → mejora

No hay catálogo general de requisitos

¿Qué capacidad falta?

Requisitos de seguridad formalizados

No hay threat modeling obligatorio

¿Qué actividad falta?

Incorporarlo en diseño

API confía en datos del cliente

¿Qué revela?

Riesgo de diseño / autorización

No hay SAST/SCA obligatorio

¿Dónde fortalecer?

Construcción / verificación

Incidente sin postmortem

¿Qué falta?

Aprendizaje y mejora sistemática

Trabajo en parejas · 7 min + puesta en común · 5 min

¿Qué entregará el estudiante?

- 1 Excel T1** Análisis detallado: SSDLC, selección SAMM, evidencias, brechas, conclusiones y fuentes.
- 2 Informe Word** Síntesis argumentada de los principales resultados.

Ambos documentos deben contar la misma historia.

Estructura del informe Word

1. Introducción
2. Análisis del caso
3. Diagnóstico OWASP SAMM
4. SSDLC
5. Brechas identificadas
6. Propuestas de mejora
7. Conclusiones
8. Referencias

Máximo 6 páginas EN TOTAL: portada + contenido + referencias.

Lo que NO se debe hacer

- No copiar todo el Excel al Word.
- No inventar vulnerabilidades o evidencias.
- No fabricar resultados de Semgrep, ZAP u otras herramientas.
- No convertir la T1 en un laboratorio técnico.
- No seleccionar prácticas sin justificar con el caso.
- No usar Shift-Left como palabra decorativa.
- No olvidar que son 2 Business Functions y 6 Practices.

¿Qué significa hacer una buena T1?

Análisis

Comprende el caso y selecciona evidencia relevante.

SAMM

2 Business Functions + 6 Practices correctamente justificadas.

SSDLC

Ubica actividades y explica relaciones.

Mejoras

Propone acciones concretas y conectadas con brechas.

Criterio central: calidad del razonamiento, no cantidad de páginas.

Exit Ticket

- 1 ¿Qué es SSDLC con tus propias palabras?
- 2 ¿Qué diferencia hay entre evidencia y opinión?
- 3 ¿Qué significa Shift-Left?
- 4 ¿Para qué usaremos OWASP SAMM en la T1?
- 5 Completa: EVIDENCIA → _____ → BRECHA → _____

3–5 minutos · individual

Lecturas sugeridas

- 1** **NIST (2022)** Secure Software Development Framework (SSDF) v1.1 — SP 800-218
- 2** **OWASP Foundation** OWASP SAMM — Software Assurance Maturity Model
- 3** **OWASP Foundation** OWASP Proactive Controls

Publicar después de la sesión las instrucciones oficiales y los archivos de la T1.

Primero aprendemos a mirar.
Luego aprendemos a medir.
Finalmente aprendemos a mejorar.



Referencias bibliográficas

Universidad Mariano Gálvez de Guatemala. (2026). Guía didáctica: Seguridad en Aplicaciones 093-013, MASI 2026. Dirección de Posgrados FISI.

OWASP Foundation. (2024). OWASP Software Assurance Maturity Model (SAMM).

Scarfone, K., Souppaya, M., & Dodson, D. (2022). Secure Software Development Framework (SSDF) Version 1.1 (NIST SP 800-218). National Institute of Standards and Technology.

OWASP Foundation. (2024). OWASP Top 10 Proactive Controls.

Microsoft. (n.d.). Microsoft Security Development Lifecycle (SDL).

BSIMM. (n.d.). Building Security In Maturity Model (BSIMM).